

Multitone FLAC 8-bit Level-Occupancy Analyzer — Implementation Plan

For Hermes: Use subagent-driven-development skill to implement this plan task-by-task.

Goal: A Python program that reads the attached FLAC (metadata + PCM audio, channel by channel), plots the waveform, derives an 8-bit quantized version of the signal, counts how often each of the 256 quantized levels is encountered, and reports the estimated time / time-fraction spent on every level over the full 1 s duration.

Architecture: One importable module multitone/analyze_multitone.py with pure, unit-testable functions (filename parser, FLAC loader, exact integer-domain 8-bit quantizer, occupancy counter, error/SQNR statistics, matplotlib plots) plus an argparse CLI. Tests follow the repo convention in root tests/.

Tech Stack: Python 3.13 (.venv), numpy 2.5.1, soundfile 0.14.0 (libsndfile FLAC decode), matplotlib 3.11.1, pytest 9.1.1. All confirmed installed in .venv.

Environment note (critical): always run project code with `env -u PYTHONPATH .venv/Scripts/python ...` — the agent shell's PYTHONPATH points at the hermes-agent cp311 venv whose numpy 2.4.3 breaks numpy imports under the cp313 project venv.

Ground truth (measured on the attached file — use as acceptance criteria)

File: multitone/audio/PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200][Nfr 031]-[96000-24]-[1s]-[8.7282]-[RD].flac (233.9 KB)

Property	Value
Format	FLAC, PCM_24, left-justified in s32 (low 8 bits zero — verified), mono , no Vorbis comment tags
Rate / length	96 000 Hz, 96 000 frames, 1.000000 s
24-bit sample range	min = −8 150 605, max = +8 150 605 (→ peak −0.250 dBFS, matches '[−0.25dB]' tag)
Signal RMS	0.355706 → −8.98 dBFS
8-bit level span occupied	−124 ... +124 → 249 / 256 levels (top 3 levels per side unreachable: peak 0.97163 < 124.5·D = 0.97266)
Most occupied level	−8: 816 samples (0.850 % = 8.500 ms); level 0: 806 (0.840 %)
ΣN_k	96 000 (exact)
$\max \setminus \{ \}$	$e \setminus \{ \}$
SQNR (8-bit)	43.96 dB
Tie samples (negative, where round-half-up $\neq$ round-half-away)	0 — both conventions bit-identical on this file

The mathematics

Audience: experts in signal processing / numerical analysis. Notation: $N = 96,000$ samples, $T_s = F_s^{-1} = 1/96,000$ s, total duration $T = NT_s = 1$ s.

1. Signal and storage model

The file renders $x[n] = \sum_{k=1}^{31} a_k \cos(2\pi f_k n / F_s + \varphi_k)$, $f_k \in [20, 43,200]$ Hz (filename Nfr $\hookrightarrow$ 031), peak-normalized to -0.25 dBFS with round-to-nearest (the [RD] tag, consistent with multitone/synthesize.py). 24-bit two's-complement storage: $m[n] \in \{-2^{23}, \dots, 2^{23} - 1\}$, normalized $\hat{x}[n] = m[n] 2^{-23} \in [-1, 1)$.

FLAC stores PCM_24 left-justified in a 32-bit container (low 8 bits zero — verified on this file), so soundfile with dtype="int32" returns that layout and the loader right-shifts by 8. The peak sample $m_{\max} = 8,150,605$ gives $20 \log_{10}(8,150,605/2^{23}) = -0.250$ dBFS, matching the filename — a built-in consistency check between metadata and data.

2. The 8-bit mid-tread uniform quantizer

256 reconstruction levels $\ell_k = kD$ for $k \in \{-128, \dots, 127\}$ with step $D = 2/256 = 2^{-7}$ (mid-tread: top level $127D$, bottom $-128D = -1$ — the standard two's-complement asymmetry). Decision regions $R_k = [(k - \frac{1}{2})D, (k + \frac{1}{2})D)$, the last closed on the right; $\mathcal{Q}(\hat{x}) = \ell_k \iff \hat{x} \in R_k$.

In sample units the step is $D \cdot 2^{23} = 2^{16}$, so the quantizer is **exact in the integer domain**:

$$q[n] = \left\lfloor \frac{m[n] + 2^{15}}{2^{16}} \right\rfloor \quad (\text{i.e. } (m + 2^{15}) // 2^{16} \text{ with integer floor division}),$$

clipped to $[-128, 127]$ (defensive; $|m| \leq 2^{23}$ already guarantees $q \geq -128$, and only $m > 127.5 \cdot 2^{16}$ would need the upper clip).

Tie conventions. Ties occur at $m = (2j + 1) 2^{15}$ (exactly half a step). The floor formula implements *round-half-up* (ties toward $+\infty$); *round-half-away-from-zero* differs only for **negative** ties. This file contains 0 negative tie samples, so both conventions are bit-identical here (verified) — the plan pins half-up and guards the equivalence with a test.

3. Quantization error and SQNR

$e[n] = \hat{x}[n] - q[n]D \in [-D/2, D/2]$; measured $\max |e| = 3.90613 \times 10^{-3} < D/2 = 3.90625 \times 10^{-3}$.

Classical result: if e is (approximately) uniform on $[-D/2, D/2]$ and uncorrelated with $\hat{x}$, then $\mathbb{E}[e^2] = D^2/12$, giving a noise floor $10 \log_{10}(D^2/12) = -52.8$ dBFS. For a full-scale sine, $\sigma_x^2 = (2^{N_b-1}D)^2/2$ yields the textbook SQNR $\approx 6.02N_b + 1.76 = 49.92$ dB at $N_b = 8$ (the $6.02 = 10 \log_{10} 4$ dB/bit slope; the $+1.76$ dB is $10 \log_{10} 6$, the ratio of sine variance to $D^2/12$).

Here the signal is a 31-tone sum, not a full-scale sine: RMS -8.98 dBFS, hence SQNR = $10 \log_{10}(\sum_n \hat{x}^2 / \sum_n e^2) \approx -8.98 + 52.8 \approx 43.9$ dB — measured **43.96 dB**, confirming both the error model and the arithmetic.

4. Level occupancy as a time estimate

$$N_k = \sum_{n=0}^{N-1} \mathbf{1}\{\mathcal{Q}(\hat{x}[n]) = k\}$$

computed with np.bincount (single $O(N)$ pass, exact integer counts). Each sample represents the interval $[nT_s, (n+1)T_s)$, so the **dwelt-time estimate** and **time-fraction estimate** are

$$\hat{t}_k = N_k T_s = \frac{N_k}{F_s}, \quad \hat{p}_k = \frac{\hat{t}_k}{T} = \frac{N_k}{N}, \quad \sum_k \hat{t}_k = T \text{ (exact)}.$$

Interpretation: $\hat{p}_k$ is the plug-in (empirical) estimator of $p_k = P(\mathcal{Q}(X) = k)$ for X a uniformly drawn sample of the signal. If samples were i.i.d., $N_k \sim \text{Binomial}(N, p_k)$ and the 1σ error of $\hat{t}_k$ would be $\sqrt{p_k(1-p_k)/N} T$; for the most occupied level ($\hat{p} = 8.50 \times 10^{-3}$, $N_k = 816$) that is $\approx 3.0 \times 10^{-4} \text{ s} = 0.30 \text{ ms}$. For a deterministic multitone the samples are strongly correlated, so this binomial SE is only nominal — the real discretization error is addressed in §7.

5. The occupancy histogram as a pdf estimator

If the underlying continuous amplitude has pdf f , then by the midpoint rule

$$p_k = \int_{(k-\frac{1}{2})D}^{(k+\frac{1}{2})D} f(u) du = D f(kD) + O(D^2 \max |f''|),$$

so $\hat{p}_k/D$ is a histogram pdf estimate at the level centres — the occupancy plot carries both readings (time on the primary axis, pdf estimate on the twin axis). Two checks on this file:

- **Gaussian cross-check:** the 31-tone sum is approximately Gaussian (CLT) with $\sigma = \text{rms}(\hat{x}) = 0.3557$, so $p_0 \approx D/(\sigma\sqrt{2\pi}) = 0.876\%$ vs measured 0.840% — the $\sim 4\%$ gap is expected (a convolution of 31 arcsine laws, not exactly Gaussian).
- **Deterministic reachability:** the occupied span is exactly $[-124, 124]$ because peak $0.97163 < (124 + \frac{1}{2})D = 0.97266$: the top 3 levels per side are unreachable — a consequence of the -0.25 dBFS normalization, not a statistic.

6. Numerical-analysis notes

- **Exactness.** The only rounding in the entire pipeline is the final division N_k/N (relative error $\lesssim 2^{-53}$, twelve orders of magnitude below the binomial SE). $m/2^{23}$ and $m/2^{16}$ are exact in binary floating point (power-of-two scaling); the integer quantizer and bincount involve no rounding at all.
- **Overflow.** $m + 2^{15}$ would overflow int32 only if $|m| > 2^{31} - 2^{15}$ — impossible at 24-bit, but the code promotes to int64 anyway; the float64 path is exact up to 2^{53} .
- **Complexity.** $O(ND)$ time, $O(256)$ extra memory; $N = 96,000$ is trivial.
- **Determinism.** No RNG, no sorts; outputs are bit-reproducible.
- **Testable invariants** (all asserted in the test suite): $\sum_k N_k = N$; $\max |e| \leq D/2$; $0 \leq \hat{p}_k \leq 1$; level span $\subseteq [-128, 127]$; CDF monotone; half-up $\equiv$ half-away on this file.

7. Caveat — sample resolution vs continuous dwell time

$\hat{t}_k = N_k T_s$ is the time fraction at **sample resolution**: it counts intervals whose *sample value* falls in R_k . The true continuous dwell time of the waveform in R_k differs by $O(T_s \max |\dot{x}|)$ per boundary crossing; with the top tone at 43.2 kHz (just under Nyquist at 48 kHz), $\max |\dot{x}|$ is large, so the count is a consistent time-average of the indicator $\mathbf{1}\{\mathcal{Q}(x) = k\}$ but not an exact per-crossing dwell time. Continuous dwell (interpolating the zero crossings of $\hat{x}(t) - \partial R_k$) is a natural extension — out of scope, noted for the future.

Program design

```
multitone/analyze_multitone.py    # the program (importable functions + main())
tests/test_analyze_multitone.py  # unit + integration tests
multitone/analysis_out/          # generated: waveform.png, occupancy_ch*.png,
    ↪ occupancy_ch*.csv
```

Function	Contract
'parse_filename(name) -> dict'	regex over the bracketed parameter string; keys 'name, nch, frac, lfl, fmin, fmax, nfr, fs, sab, dur, vol, seed, score, round'
'load_flac(path) -> FlacSignal'	'sf.info' + 'sf.read(dtype="int32", always_2d=True)'; 'raw = x » 8' (int64, true 24-bit values); supports PCM_16 / PCM_24; per-channel access via 'sig.channel(c)'
'quantize_8bit(m, sab=24) -> int8'	$q = (m + 2^{sab-9}) // 2^{sab-8}$, clipped to $[-128, 127]$; exact integer arithmetic, round-half-up
'level_occupancy(q) -> (counts, levels)'	'np.bincount(q+128, minlength=256)' → (int64[256], int8[256])
'dwell_times(counts, sr) -> float64[256]'	$N_k / F_s \text{ seconds}$
'quant_error_stats(m, q, sab) -> dict'	'max_abs_err', 'half_step', 'sqnr_db'
'plot_waveform(sig, q_by_ch, out_dir) -> Path'	per channel: full-duration row + 4 ms zoom row with the 8-bit staircase overlay
'plot_occupancy(counts, levels, sr, ch, out_dir) -> Path'	256-bar time chart + twin-axis pdf estimate $\hat{p}_k / D$
'main(argv=None) -> int'	argparse CLI; console report per channel; full 256-row CSV per channel

CLI:

```
env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone [FLAC_PATH]
    ↪ [--out DIR] [--no-plot]
```

(default FLAC_PATH = the attached file; default --out multitone/analysis_out/)

"Channel by channel": the loader returns a (frames, channels) int64 array; the analysis loop, the waveform plot rows, the occupancy figures, and the CSVs are all per-channel. The attached file is mono, so everything below is ch 0; stereo files in the repo are handled by the same loop.

Tasks

Task 1: Filename parser

Objective: parse the structured FLAC filename into a metadata dict.

Files:

- Create: multitone/analyze_multitone.py
- Create: tests/test_analyze_multitone.py

Step 1: Write failing test — tests/test_analyze_multitone.py:

```
from multitone.analyze_multitone import parse_filename

def test_parse_filename():
    md = parse_filename(
        "PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200][Nfr 031]"
        "-[96000-24]-[1s]-[-0.25dB]-[057923]-[ 8.7282]-[RD].flac")
    assert md == {"name": "PrimeToneNewPhi", "nch": 1, "frac": 3, "lfl": 43,
                  "fmin": 20, "fmax": 43200, "nfr": 31, "fs": 96000, "sab": 24,
                  "dur": 1, "vol": -0.25, "seed": 57923, "score": 8.7282,
                  "round": "RD"}
```

Step 2: Run test to verify failure

Run: env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py::test_parse_file

↪ -v Expected: FAIL — ModuleNotFoundError: multitone.analyze_multitone

Step 3: Write minimal implementation — multitone/analyze_multitone.py:

```
"""Read a multitone FLAC (metadata + PCM, channel by channel), plot the
waveform, and estimate the time spent on each 8-bit quantization level.

Usage:
    env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone [path]
"""
from __future__ import annotations
import re

_NAME_RE = re.compile(
    r"^(?P<name>[A-Za-z0-9]+)"
    r" $${Nch}s*(?P<nch>\d+)$ $${Frac}s*(?P<frac>\d+)$ $${LFL}s*(?P<lfl>\d+)$ $ "
    r" $${(?P<fmin>\d+)-(?P<fmax>\d+)} $${Nfr}s*(?P<nfr>\d+)$ $ "
    r"- $${(?P<fs>\d+)-(?P<sab>\d+)} $ - $${(?P<dur>\d+)s}$ - "
    ↪ $${(?P<vol>-?\d+(?:\.\d+)?)dB}$ "
    r"- $${(?P<seed>\d+)} $ - $${s*(?P<score>\d+(?:\.\d+)?)}$ - $${(?P<round>[A-Z]+)}$ "
    ↪ \.flac$"
)

def parse_filename(name: str) -> dict:
    m = _NAME_RE.match(name)
    if not m:
        raise ValueError(f"unrecognized multitone filename: {name!r}")
    d = m.groupdict()
    for k in ("nch", "frac", "lfl", "fmin", "fmax", "nfr", "fs", "sab",
              "dur", "seed"):
        d[k] = int(d[k])
    d["vol"] = float(d["vol"])
    d["score"] = float(d["score"])
    return d
```

Step 4: Run test to verify pass — same command → PASS Step 5: Commit

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): filename parameter parser"
```

Task 2: FLAC loader (metadata + channel-by-channel PCM)

Objective: read stream metadata and all channels into an int64 array of true bit-depth values.

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing test

```
from pathlib import Path

import numpy as np
import pytest

from multitone.analyze_multitone import load_flac

FLAC = Path(
    "multitone/audio/PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200]"
    "[Nfr 031]-[96000-24]-[1s]-[-0.25dB]-[057923]-[ 8.7282]-[RD].flac")

def test_load_flac_ground_truth():
    if not FLAC.exists():
        pytest.skip("attached flac not present")
    sig = load_flac(FLAC)
    assert (sig.samplerate, sig.frames, sig.channels, sig.bits) == (
        96000, 96000, 1, 24)
    m = sig.channel(0)
    assert m.dtype == np.int64 and m.shape == (96000,)
    assert m.min() == -8150605 and m.max() == 8150605
    assert abs(20 * np.log10(m.max() / 2**23) + 0.25) < 0.005 # -0.25 dBFS
    assert sig.metadata["fs"] == 96000 and sig.metadata["sab"] == 24
```

Step 2: Run test to verify failure → FAIL (ImportError: load_flac)

Step 3: Write minimal implementation (append to multitone/analyze_multitone.py):

```
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
import soundfile as sf

@dataclass(frozen=True)
class FlacSignal:
    path: Path
    samplerate: int
    channels: int
    frames: int
    duration: float
    subtype: str
    bits: int # true bit depth from the stream (16 or 24)
    raw: np.ndarray # int64, shape (frames, channels)
    metadata: dict = field(default_factory=dict)

    def channel(self, c: int) -> np.ndarray:
        return self.raw[:, c]

def load_flac(path) -> FlacSignal:
    path = Path(path)
```

```

info = sf.info(str(path))
bits = {"PCM_24": 24, "PCM_16": 16}.get(info.subtype)
if bits is None:
    raise ValueError(f"unsupported FLAC subtype: {info.subtype}")
x, sr = sf.read(str(path), dtype="int32", always_2d=True)
raw = (x >> 8).astype(np.int64) if bits == 24 else x.astype(np.int64)
return FlacSignal(path=path, samplerate=sr, channels=info.channels,
                  frames=info.frames, duration=float(info.duration),
                  subtype=info.subtype, bits=bits, raw=raw,
                  metadata=parse_filename(path.name))

```

Step 4: Run test to verify pass → PASS Step 5: Commit

```

git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): FLAC loader (metadata + channel-by-channel PCM)"

```

Task 3: 8-bit quantizer (exact integer domain)

Objective: the mid-tread quantizer of §2, with boundary and tie-convention tests.

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing tests

```

from multitone.analyze_multitone import quantize_8bit

def test_quantize_boundaries():
    assert quantize_8bit(np.array([0]))[0] == 0
    assert quantize_8bit(np.array([2**15 - 1]))[0] == 0      # just below 0/1 boundary
    assert quantize_8bit(np.array([2**15]))[0] == 1           # positive tie: up ==
    ↪away
    assert quantize_8bit(np.array([-2**15]))[0] == 0           # negative tie: up
    ↪(toward +inf)
    assert quantize_8bit(np.array([-2**15 + 1]))[0] == 0
    assert quantize_8bit(np.array([2**23 - 1]))[0] == 127     # beyond top boundary:
    ↪clip
    assert quantize_8bit(np.array([-2**23]))[0] == -128        # bottom level, exact

def test_tie_conventions():
    # half-up (floor) vs half-away-from-zero differ only on negative ties
    m = np.array([-2**15, 2**15, 2**15 + 1, -2**15 - 1], dtype=np.int64)
    q_up = quantize_8bit(m)
    q_away = np.where(m >= 0, (m + 2**15) // 2**16, -((-m + 2**15) // 2**16))
    assert np.array_equal(q_up, [0, 1, 1, -1])
    assert np.array_equal(q_away, [-1, 1, 1, -1])
    assert int((q_up != q_away).sum()) == 1 and bool((q_up != q_away)[0])

def test_quantizer_matches_float_definition():
    # integer formula must equal the textbook Q(x) = D * floor(x/D + 1/2)
    m = np.arange(-2**23, 2**23, 997, dtype=np.int64)
    x = m / 2.0**23
    q_ref = np.clip(np.floor(x / (2.0 / 256.0) + 0.5), -128, 127).astype(np.int8)
    assert np.array_equal(quantize_8bit(m), q_ref)

```

Step 2: Run tests to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```
def quantize_8bit(m: np.ndarray, sab: int = 24) -> np.ndarray:
    """Map two's-complement samples to 8-bit mid-tread levels [-128, 127].

    q = floor(m / 2^(sab-8) + 1/2), computed in the integer domain (exact,
    round-half-up), clipped to the 8-bit range (defensive).
    """
    if sab not in (16, 24):
        raise ValueError(f"unsupported bit depth: {sab}")
    m = np.ascontiguousarray(m, dtype=np.int64)
    d = 1 << (sab - 8)          # step in sample units (2^16 for 24-bit)
    q = (m + d // 2) // d       # floor division == floor(m/d + 1/2), exact
    return np.clip(q, -128, 127).astype(np.int8)
```

Step 4: Run tests to verify pass → 3 PASS Step 5: Commit

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): exact integer 8-bit mid-tread quantizer"
```

Task 4: Level occupancy and dwell times

Objective: the counts N_k , levels, and $\hat{t}_k = N_k / F_s$.

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing test

```
from multitone.analyze_multitone import dwell_times, level_occupancy

def test_occupancy_invariants():
    rng = np.random.default_rng(0)
    m = rng.integers(-2**23, 2**23, size=100_000, dtype=np.int64)
    q = quantize_8bit(m)
    counts, levels = level_occupancy(q)
    assert counts.dtype == np.int64 and counts.shape == (256,)
    assert counts.sum() == m.size
    assert levels[0] == -128 and levels[-1] == 127
    t = dwell_times(counts, 96000)
    assert np.all(t >= 0)
    assert t.sum() == pytest.approx(m.size / 96000)
```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```
def level_occupancy(q: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """counts[k] = number of samples on level k-128; (int64[256], int8[256])."""
    counts = np.bincount(q.astype(np.int64) + 128, minlength=256)
    return counts, np.arange(-128, 128, dtype=np.int8)

def dwell_times(counts: np.ndarray, samplerate: int) -> np.ndarray:
    """t_k = N_k / F_s : estimated seconds spent on each level."""
    return counts.astype(np.float64) / float(samplerate)
```

Step 4: Run test to verify pass → PASS Step 5: Commit

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): level occupancy + dwell-time estimator"
```


Task 5: Quantization error statistics

Objective: $\max |e|$, the $D/2$ bound, and the SQNR (with a full-scale-sine reference check).

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing test

```
from multitone.analyze_multitone import quant_error_stats

def test_error_bound_and_snr_095fs_sine():
    # 0.95 FS sine: no clipping, error strictly inside [-D/2, D/2];
    # SQNR = 10*log10(6) + 20*log10(0.95 * 128) = 49.48 dB (uniform-error model)
    n = 96000
    m = np rint(0.95 * 2**23 * np.sin(2 * np.pi * 440 * np.arange(n) / 96000)
                ).astype(np.int64)
    q = quantize_8bit(m)
    st = quant_error_stats(m, q, 24)
    assert st["max_abs_err"] <= st["half_step"]
    assert abs(st["snr_db"] - 49.48) < 1.0
```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```
def quant_error_stats(m: np.ndarray, q: np.ndarray, sab: int = 24) -> dict:
    scale = float(1 << (sab - 1))
    d = float(1 << (sab - 8))
    x = m.astype(np.float64) / scale
    e = x - q.astype(np.float64) * (d / scale)
    return {
        "max_abs_err": float(np.abs(e).max()),
        "half_step": 0.5 * d / scale,
        "snr_db": float(10.0 * np.log10(np.sum(x * x) / np.sum(e * e))),
    }
```

Step 4: Run test to verify pass → PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): quantization error / SQNR statistics"
```

Task 6: Plots (waveform + occupancy)

Objective: the two figures — full/zoom waveform with the 8-bit staircase overlay, and the 256-level occupancy chart with the pdf-estimate twin axis.

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing test

```
from multitone.analyze_multitone import plot_occupancy, plot_waveform

def test_plots_created(tmp_path):
    if not FLAC.exists():
        pytest.skip("attached flac not present")
    sig = load_flac(FLAC)
```

```

m = sig.channel(0)
q = quantize_8bit(m)
counts, levels = level_occupancy(q)
p1 = plot_waveform(sig, {0: q}, tmp_path)
p2 = plot_occupancy(counts, levels, sig.samplerate, 0, tmp_path)
assert p1.exists() and p1.stat().st_size > 10_000
assert p2.exists() and p2.stat().st_size > 10_000

```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation (append to multitone/analyze_multitone.py)

```

def _agg():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    return plt

def plot_waveform(sig: FlacSignal, q_by_channel: dict, out_dir) -> Path:
    plt = _agg()
    t = np.arange(sig.frames) / sig.samplerate
    scale = 2.0 ** (sig.bits - 1)
    D = 2.0 / 256.0
    nzm = min(384, sig.frames) # 4 ms zoom at 96 kHz
    fig, axes = plt.subplots(2 * sig.channels, 1,
                             figsize=(11, 2.2 * sig.channels), sharex=True)
    axes = np.atleast_1d(axes)
    for c in range(sig.channels):
        full = sig.channel(c) / scale
        axes[2 * c].plot(t, full, lw=0.3, color="navy")
        axes[2 * c].set_ylabel(f"ch{c} full")
        axes[2 * c].set_ylim(-1.05, 1.05)
        axes[2 * c + 1].plot(t[:nzm], full[:nzm], lw=0.4,
                             color="navy", label="original")
        axes[2 * c + 1].step(t[:nzm], q_by_channel[c][:nzm] * D,
                             where="mid", lw=0.9, color="crimson",
                             label="8-bit quantized")
        axes[2 * c + 1].set_ylabel(f"ch{c} 4 ms zoom")
        axes[2 * c + 1].legend(loc="upper right", fontsize=8)
    axes[-1].set_xlabel("time (s)")
    fig.suptitle(f"{sig.path.name} - {sig.samplerate} Hz, "
                 f"{sig.frames} fr, {sig.channels} ch")
    fig.tight_layout(rect=(0, 0, 1, 0.98))
    out = Path(out_dir) / "waveform.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    return out

def plot_occupancy(counts: np.ndarray, levels: np.ndarray, samplerate: int,
                   channel: int, out_dir) -> Path:
    plt = _agg()
    t = dwell_times(counts, samplerate)
    p = counts / counts.sum()
    fig, ax = plt.subplots(figsize=(11, 4))
    ax.bar(levels, t, width=1.0, color="steelblue")
    ax.set_xlabel("8-bit level $k$")
    ax.set_ylabel("time $t_k$ (s)")
    ax.set_title(f"Level occupancy, ch{channel}: {int(counts.sum())} samples, "
                 f"{int(np.count_nonzero(counts))}/256 levels occupied")
    ax2 = ax.twinx()
    ax2.plot(levels, p * 128.0, "r.", ms=3,

```

```

        label=r"$\hat{p}_k / D$ (pdf estimate)")
    ax2.set_ylabel(r"$\hat{p}_k / D$ (pdf estimate)")
    ax2.legend(loc="upper right", fontsize=8)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    out = Path(out_dir) / f"occupancy_ch{channel}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    return out

```

Step 4: Run test to verify pass → PASS Step 5: Commit

```

git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): waveform + occupancy plots"

```

Task 7: CLI and console report

Objective: main() entry point — per-channel report, top-10 level table, full 256-row CSV, plots.

Files:

- Modify: multitone/analyze_multitone.py
- Modify: tests/test_analyze_multitone.py

Step 1: Write failing test

```

from multitone.analyze_multitone import main

def test_cli(tmp_path, capsys):
    if not FLAC.exists():
        pytest.skip("attached flac not present")
    rc = main([str(FLAC), "--out", str(tmp_path), "--no-plot"])
    out = capsys.readouterr().out
    assert rc == 0
    assert "96000" in out and "249/256" in out
    assert "0.850" in out # top level -8: 816 samples
    csv = tmp_path / "occupancy_ch0.csv"
    assert csv.exists()
    assert len(csv.read_text().strip().splitlines()) == 257 # header + 256

```

Step 2: Run test to verify failure → FAIL (ImportError: main)

Step 3: Write minimal implementation (append to multitone/analyze_multitone.py)

```

import argparse

DEFAULT_FLAC = (Path(__file__).resolve().parent / "audio" /
    "PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200]"
    "[Nfr 031]-[96000-24]-[1s]-[-0.25dB]-[057923]-[ 8.7282]-[RD].flac")

def main(argv=None) -> int:
    ap = argparse.ArgumentParser(
        description="FLAC 8-bit level-occupancy analyzer")
    ap.add_argument("path", nargs="?", default=str(DEFAULT_FLAC))
    ap.add_argument("--out",
        default=str(Path(__file__).resolve().parent / "analysis_out"))
    ap.add_argument("--no-plot", action="store_true")
    a = ap.parse_args(argv)

    sig = load_flac(Path(a.path))

```

```

out_dir = Path(a.out)
out_dir.mkdir(parents=True, exist_ok=True)
if sig.metadata.get("sab") != sig.bits:
    print(f"WARNING: filename sab={sig.metadata.get('sab')} "
          f"!= stream bits={sig.bits}")

print(f"file: {sig.path.name}")
print(f" {sig.samplerate} Hz, {sig.frames} frames, {sig.channels} ch, "
      f"{sig.subtype}, {sig.duration:.6f} s")
print(f" params: {sig.metadata}")

q_all = {}
for c in range(sig.channels):
    m = sig.channel(c)
    q = quantize_8bit(m, sig.bits)
    q_all[c] = q
    counts, levels = level_occupancy(q)
    st = quant_error_stats(m, q, sig.bits)
    scale = float(2.0 ** (sig.bits - 1))
    peak = m.max() / scale
    rms = float(np.sqrt(np.mean((m / scale) ** 2)))
    occ = np.nonzero(counts)[0]
    print(f"channel {c}: peak {20*np.log10(peak):+.3f} dBFS, "
          f"rms {20*np.log10(rms):+.2f} dBFS, "
          f"levels {occ.size}/256 [{int(levels[occ[0]])}, "
          f"{int(levels[occ[-1]])}], SQNR {st['sqnr_db']:.2f} dB, "
          f"max|e| {st['max_abs_err']:.9f} (<= {st['half_step']:.9f})")
    top = sorted(np.argsort(counts)[::-1][:10])
    print(" top-10 levels (level, samples, ms, %):")
    for k in top:
        print(f" {int(levels[k]):+4d} {counts[k]:6d} "
              f"{dwell_times(counts, sig.samplerate)[k]*1e3:8.3f} "
              f"{counts[k]/m.size*100:6.3f}")
    np.savetxt(out_dir / f"occupancy_ch{c}.csv",
               np.column_stack([levels, counts,
                               dwell_times(counts, sig.samplerate)]),
               delimiter=",", header="level,samples,seconds", comments="")
if not a.no_plot:
    print(f"plot: {plot_waveform(sig, q_all, out_dir)}")
    for c in range(sig.channels):
        counts, levels = level_occupancy(q_all[c])
        print(f"plot: {plot_occupancy(counts, levels, sig.samplerate, c,
        ↪out_dir)}")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Step 4: Run test to verify pass → PASS Step 5: Commit

```

git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): CLI + console report + per-channel CSV"

```

Task 8: End-to-end regression against ground truth

Objective: pin the measured values of the attached file (ground-truth table above) as a regression test, then run the real CLI.

Files:

- Modify: tests/test_analyze_multitone.py

Step 1: Write the test

```
def test_end_to_end_attached_file():
    if not FLAC.exists():
        pytest.skip("attached flac not present")
    sig = load_flac(FLAC)
    m = sig.channel(0)
    q = quantize_8bit(m, 24)
    counts, levels = level_occupancy(q)
    assert counts.sum() == 96000
    occ = np.nonzero(counts)[0]
    assert len(occ) == 249
    assert int(levels[occ[0]]) == -124 and int(levels[occ[-1]]) == 124
    assert counts[120] == 816          # level -8, most occupied
    assert counts[128] == 806          # level 0
    st = quant_error_stats(m, q, 24)
    assert st["max_abs_err"] <= st["half_step"]
    assert 43.5 < st["sqnr_db"] < 44.5
    # rounding-convention equivalence on this file (no negative ties)
    assert int(((m < 0) & ((m & 0xFFFF) == 0x8000)).sum()) == 0
    q_away = np.where(m >= 0, (m + 2**15) // 2**16, -((-m + 2**15) // 2**16))
    assert np.array_equal(q, np.clip(q_away, -128, 127).astype(np.int8))
```

Step 2: Run the full suite

Run: `env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py`
↪ -v Expected: 10 passed (0 skipped, since the FLAC is present)

Step 3: Run the real CLI and inspect outputs

```
env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone
```

Expected console output (must match the ground-truth table): 96000 Hz, 96000 frames,
↪ 1 ch; levels 249/256 [-124, 124]; peak -0.250 dBFS; rms -8.98 dBFS; SQNR 43.96 dB;
max|e| 0.003906130 (<= 0.003906250); top level -8 816 8.500 0.850; level +0 806 8.396 0.840.
Then visually check multitone/analysis_out/waveform.png (full trace + 4 ms zoom showing
the staircase hugging the waveform) and occupancy_ch0.png (256 bars, mass concentrated
near the centre, empty top/bottom 3 levels per side).

Step 4: Commit

```
git add tests/test_analyze_multitone.py
git commit -m "test(multitone): end-to-end ground-truth regression for attached
↪ FLAC"
```

Acceptance criteria

- `env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py`
-v → all green (10 tests).
- CLI run on the attached file prints the ground-truth values (table above) and writes
multitone/analysis_out/ with waveform.png, occupancy_ch0.png, occupancy_ch0.csv (257
lines).
- $\sum_k N_k = 96,000$ exact; $\sum_k \hat{t}_k = 1.000000$ s.
- Program is generic in channel count (the per-channel loop handles stereo files in the
same folder without changes).

Risks, tradeoffs, open questions

- **Tie convention.** Implementation pins round-half-up. If the generator's [RD] mode used a different negative-tie break, counts on tie levels could differ by 1. This file has 0 negative ties, so it is moot here; Task 8's equivalence test guards it for other files.
- **16-bit files.** Supported via sab (step becomes 2^8); ground-truth tests are pinned to the 24-bit file. 32-bit float FLAC is rejected with a clear error (out of scope).
- **Filename score (8.7282).** An optimizer score — parsed but unused (YAGNI).
- **LFL / Frac semantics.** Parsed as opaque integers; their meaning lives in `multitone/producer.py/synthesize.py` and is not needed for the analysis.
- **Out of scope (noted extensions):** continuous (interpolated) dwell times, spectral analysis, cross-channel analysis of stereo pairs.